PROCESO DE SELECCIÓN · ÁREA DE TECNOLOGÍA

Prueba técnica: Subasta en vivo

Reto de desarrollo con Python y JavaScript

Dato	Detalle
Empresa	E-HUK
Stack obligatorio	Backend en Python · Frontend en JavaScript (todo lo demás es libre)
Tiempo de trabajo	Máximo 4 horas
Plazo de entrega	2 días
Entrega	Repositorio en GitHub
Reunión posterior	En persona: explicas tu solución y tus decisiones

El reto

Construye una aplicación web de **subastas en vivo**. Varias personas ven la misma subasta al mismo tiempo, pujan, y todas ven lo que pasa al instante, hasta que el tiempo se acaba y hay un ganador.

Reglas del juego

- **Backend en Python y frontend en JavaScript.** Todo lo demás lo eliges tú: framework, base de datos, cómo comunicar el tiempo real, librerías y estructura.
- **Puedes usar IA.** Después de la entrega nos sentaremos contigo en persona para que **nos expliques todo**: cómo lo resolviste, qué tecnologías elegiste y por qué, y qué no resolviste. Entrega solo lo que puedas explicar.
- No hay login: el postor se identifica con un nombre de texto.
- Si algo del enunciado es ambiguo, **decide tú**. Anota las decisiones para contárnoslas en la reunión.
- El alcance es exigente a propósito. **Priorizar es parte de la prueba.** Los niveles indican el orden: lo del Nivel 1 es obligatorio.
- En el repositorio solo necesitamos un `README` con los **pasos exactos para correrlo**. Lo demás nos lo cuentas en persona.
- Nunca subas claves ni secretos al repositorio.

Condiciones de entrega

Tiempo de trabajo: máximo 4 horas. Tienes 2 días para entregar.

Entrega: repositorio en GitHub (público o con acceso para nosotros), con commits frecuentes y mensajes descriptivos.

Tareas y Criterios de Aceptacion

Nivel 1: obligatorio

#	Criterio de aceptación
HU-1	**Crear subasta**: título (1 a 100 caracteres), descripción, precio inicial (> 0), incremento mínimo (> 0) y fecha/hora de fin (futura, máximo 7 días).
HU-2	**Listar** subastas abiertas y cerradas. **Detalle** con precio actual, mejor postor, tiempo restante e historial de pujas (más reciente primero).
HU-3	**Pujar** con nombre y monto. Es válida si: la subasta está abierta, el monto es **al menos** precio actual + incremento mínimo (la primera puja, al menos el precio inicial), y el postor **no es el mejor postor actual**. Si no es válida, se rechaza con un motivo claro y un código HTTP apropiado.
HU-4	**Dinero exacto**, con 2 decimales. Nunca errores de redondeo.
HU-5	**Cierre**: al llegar la hora de fin no se aceptan más pujas, y el ganador queda registrado. Si no hubo pujas, la subasta queda desierta. Debe ser correcto **aunque nadie esté mirando** y **aunque el servidor se reinicie**.
HU-6	**Concurrencia**: si llegan 50 pujas casi simultáneas a la misma subasta, el resultado final es consistente: el precio actual es la mayor puja válida, el historial solo tiene pujas válidas con montos estrictamente crecientes y nunca hay dos pujas aceptadas con el mismo monto. **Tienes que poder demostrarlo** (un script o una prueba).
HU-7	**Persistencia**: los datos sobreviven a un reinicio.
HU-8	**Interfaz**: lista, detalle, crear subasta y pujar. Se ven los errores. Un doble clic no envía dos pujas. Las acciones inválidas se explican a la persona, sin mostrar errores técnicos crudos.

Nivel 2: esperado

#	Criterio
HU-9	**Tiempo real**: todos los que miran la subasta ven las pujas nuevas y el cierre **sin recargar la página**.
HU-10	**Cuenta regresiva basada en el reloj del servidor**: debe ser correcta aunque el reloj del computador del usuario esté 5 minutos adelantado o atrasado.
HU-11	**Validador** si entra una puja válida en los últimos 60 segundos, el fin se extiende para que queden 60 segundos desde esa puja, hasta un máximo de 10 extensiones. La extensión y la puja son una sola operación atómica.

#	Criterio
HU-12	**Idempotencia**: si el cliente reintenta la misma puja (por mala red o doble envío), no se registra dos veces.
HU-13	**Pruebas automáticas** de la lógica de pujas, cierre y concurrencia.
HU-14	**Recuperación**: si la conexión en vivo se cae y vuelve, el cliente se actualiza solo y no se pierde ninguna puja ocurrida mientras estuvo desconectado.

Nivel 3: reto (solo si te sobra tiempo)

- **Puja automática:** el usuario fija un máximo y el sistema puja por él con el mínimo incremento necesario.
- **Precio de reserva oculto:** si no se alcanza, la subasta se declara desierta aunque haya pujas.
- **Varias instancias del backend** funcionando a la vez sin perder consistencia ni el tiempo real.
- **1000 observadores** en una sola subasta sin degradar el servicio.

Transversal

- Validación siempre en el servidor, aunque el cliente también valide.
- Errores con formato consistente.
- Protección razonable contra inyección y contra contenido malicioso en lo que escribe el usuario.

Escenarios que probaremos juntos en la reunión

1. Crear una subasta con incremento de 5.00 y pujar 10.00, luego 12.00 (debe rechazarse) y luego 15.00.
2. Pujar dos veces seguidas con el mismo nombre.
3. Dos navegadores a la vez: uno puja y el otro lo ve sin recargar.
4. Una puja en los últimos segundos y ver qué pasa con el tiempo.
5. Lanzar 50 pujas simultáneas y revisar la consistencia.
6. Apagar y encender el servidor con una subasta a punto de cerrar.
7. Cambiar la hora del computador y mirar la cuenta regresiva.
8. Escribir `<script>alert(1)</script>` como título o como nombre.
9. Cortar la red unos segundos y restablecerla.

Mucho éxito, te desea el equipo de E-HUK. Te esperamos